

VISVESVARAYA TECHNOLOGICAL UNIVERSITY

“JnanaSangama”, Belgaum -590014, Karnataka.



LAB REPORT
on

OPERATING SYSTEMS

Submitted by

UMADEVI (1BF24CS322)

in partial fulfillment for the award of the degree of
BACHELOR OF ENGINEERING
in
COMPUTER SCIENCE AND ENGINEERING



B.M.S. COLLEGE OF ENGINEERING

(Autonomous Institution under VTU)

BENGALURU-560019

Feb-2026 to June-2026

B. M. S. College of Engineering,
Bull Temple Road, Bangalore 560019
(Affiliated To Visvesvaraya Technological University, Belgaum)
Department of Computer Science and Engineering



CERTIFICATE

This is to certify that the Lab work entitled “OPERATING SYSTEMS – 23CS4PCOPS” carried out by Umadevi (1BF24CS322), who is Bonafide student of B. M. S. College of Engineering. It is in partial fulfilment for the award of Bachelor of Engineering in Computer Science and Engineering of the Visvesvaraya Technological University, Belgaum during the year Feb-2026 to June-2026 . The Lab report has been approved as it satisfies the academic requirements in respect of a OPERATING SYSTEMS - (23CS4PCOPS) work prescribed for the said degree.

Dr Seema Patil
Assistant Professor
Department of CSE
BMSCE, Bengaluru

Dr. Kavitha Sooda
Professor and Head
Department of CSE
BMSCE, Bengaluru

Index Sheet

Sl. No.	Experiment Title	Page No.
1.	Write a C program to simulate the following non-pre-emptive CPU scheduling algorithm to find turnaround time and waiting time. a)FCFS b)SJF (pre-emptive & Non-preemptive)	4-10
2.	Write a C program to simulate the following CPU scheduling algorithm to find turnaround time and waiting time. a)Priority (pre-emptive & Non-pre-emptive) b)Round Robin (Experiment with different quantum sizes for RR algorithm)	11-15
3.	Write a C program to simulate multi-level queue scheduling algorithm considering the following scenario. All the processes in the system are divided into two categories – system processes and user processes. System processes are to be given higher priority than user processes. Use FCFS scheduling for the processes in each queue.	16-18
4.	Write a C program to simulate Real-Time CPU Scheduling algorithms: a) Rate- Monotonic b) Earliest-deadline First c) Proportional scheduling	19-25
5.	Write a C program to simulate producer-consumer problem using semaphores	26-28
6.	Write a C program to simulate the concept of Dining Philosophers problem.	29-31
7.	Write a C program to simulate Bankers algorithm for the purpose of deadlock avoidance.	32-24

Course Outcomes

C01	Apply the different concepts and functionalities of Operating System
C02	Analyse various Operating system strategies and techniques
C03	Demonstrate the different functionalities of Operating System.
C04	Conduct practical experiments to implement the functionalities of Operating system.

Lab Program 1

Write a C program to simulate the following non-pre-emptive CPU scheduling algorithm to find turnaround time and waiting time.

a)FCFS

b) SJF (pre-emptive & Non-preemptive)

CODE:

a)FCFS

```
#include <stdio.h>
#include <limits.h>
#include <stdbool.h>
struct Process {
    int pid;
    int arrival;
    int burst;
    int remaining;
    int completion;
    int turnaround;
    int waiting;
};
int main() {
    int n;
    printf("Enter number of processes: ");
    scanf("%d", &n);
    struct Process p[n];
    for (int i = 0; i < n; i++) {
        printf("Enter Arrival Time and Burst Time for Process %d: ", i+1);
        p[i].pid = i+1;
        scanf("%d %d", &p[i].arrival, &p[i].burst);
        p[i].remaining = p[i].burst;
    }
    int current_time = 0, completed = 0;
    float avg_turnaround = 0, avg_waiting = 0;
    while (completed < n) {
        int idx = -1;
        int min_remaining = INT_MAX;
```

```

for (int i = 0; i < n; i++) {
    if (p[i].arrival <= current_time && p[i].remaining > 0) {
        if (p[i].remaining < min_remaining) {
            min_remaining = p[i].remaining;
            idx = i;
        }
    }
}
if (idx == -1) {
    current_time++;
} else {
    p[idx].remaining--;
    current_time++;
    if (p[idx].remaining == 0) {
        p[idx].completion = current_time;
        p[idx].turnaround = p[idx].completion - p[idx].arrival;
        p[idx].waiting = p[idx].turnaround - p[idx].burst;
        avg_turnaround += p[idx].turnaround;
        avg_waiting += p[idx].waiting;
        completed++;
    }
}
}

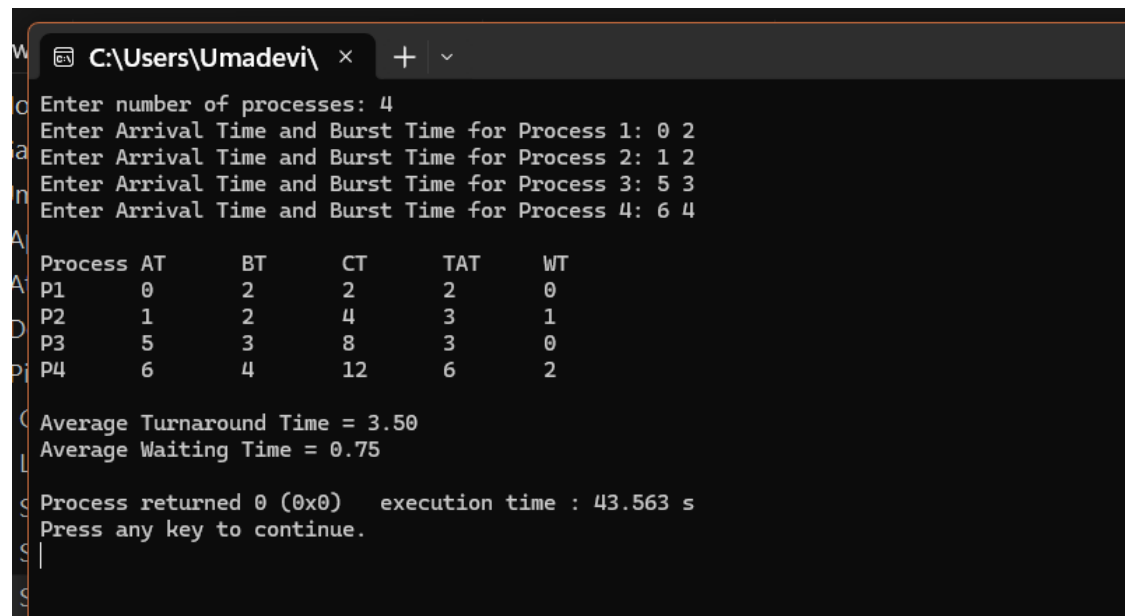
printf("\nProcess\tAT\tBT\tCT\tTAT\tWT\n");
for (int i = 0; i < n; i++) {
    printf("P%d\t%d\t%d\t%d\t%d\t%d\n",
        p[i].pid, p[i].arrival, p[i].burst,
        p[i].completion, p[i].turnaround, p[i].waiting);
}

printf("\nAverage Turnaround Time = %.2f", avg_turnaround / n);
printf("\nAverage Waiting Time = %.2f\n", avg_waiting / n);

return 0;
}

```

OUTPUT:



```
W C:\Users\Umadevi\ x + v
O
a
n
A
A
D
P
C
L
S
S
S
S

Enter number of processes: 4
Enter Arrival Time and Burst Time for Process 1: 0 2
Enter Arrival Time and Burst Time for Process 2: 1 2
Enter Arrival Time and Burst Time for Process 3: 5 3
Enter Arrival Time and Burst Time for Process 4: 6 4

Process AT    BT    CT    TAT    WT
P1      0      2      2      2      0
P2      1      2      4      3      1
P3      5      3      8      3      0
P4      6      4     12      6      2

Average Turnaround Time = 3.50
Average Waiting Time = 0.75

Process returned 0 (0x0)  execution time : 43.563 s
Press any key to continue.
```

b) SJF (pre-emptive & Non-preemptive)

SJF pre-emptive:

```
#include <stdio.h>
#include <stdbool.h>
struct Process {
    int pid;
    int at;
    int bt;
    int rt;
    int ct;
    int tat;
    int wt;
    bool finished;
};
int main() {
    int n;
    printf("Enter number of processes: ");
    scanf("%d", &n);
    struct Process p[n];
    for (int i = 0; i < n; i++) {
        p[i].pid = i + 1;
        printf("Enter arrival time and burst time of P%d: ", i + 1);
        scanf("%d %d", &p[i].at, &p[i].bt);
        p[i].rt = p[i].bt;
```

```

    p[i].finished = false;
}
int current_time = 0, completed = 0;
float total_tat = 0, total_wt = 0;
while (completed < n) {
    int idx = -1;
    int min_rt = 1e9;
    for (int i = 0; i < n; i++) {
        if (!p[i].finished && p[i].at <= current_time && p[i].rt < min_rt) {
            min_rt = p[i].rt;
            idx = i;
        }
    }
    if (idx == -1) {
        current_time++;
    } else {
        p[idx].rt--;
        current_time++;
        if (p[idx].rt == 0) {
            p[idx].ct = current_time;
            p[idx].tat = p[idx].ct - p[idx].at;
            p[idx].wt = p[idx].tat - p[idx].bt;
            total_tat += p[idx].tat;
            total_wt += p[idx].wt;
            p[idx].finished = true;
            completed++;
        }
    }
}
printf("\nSJF Preemptive Scheduling (SRTF):\n");
printf("PID\tAT\tBT\tCT\tTAT\tWT\n");
for (int i = 0; i < n; i++) {
    printf("P%d\t%d\t%d\t%d\t%d\t%d\n",
        p[i].pid, p[i].at, p[i].bt, p[i].ct, p[i].tat, p[i].wt);
}
printf("\nAverage Turnaround Time: %.2f ms", total_tat / n);
printf("\nAverage Waiting Time: %.2f ms\n", total_wt / n);
return 0;
}

```

OUTPUT:

```
C:\Users\Umadevi\ x + v
Enter number of processes: 5
Enter arrival time and burst time of P1: 2 1
Enter arrival time and burst time of P2: 1 5
Enter arrival time and burst time of P3: 4 1
Enter arrival time and burst time of P4: 0 6
Enter arrival time and burst time of P5: 2 3

SJF Preemptive Scheduling (SRTF):
PID    AT    BT    CT    TAT    WT
P1      2     1     3     1     0
P2      1     5    11    10     5
P3      4     1     5     1     0
P4      0     6    16    16    10
P5      2     3     7     5     2

Average Turnaround Time: 6.60 ms
Average Waiting Time: 3.40 ms

Process returned 0 (0x0)   execution time : 30.889 s
Press any key to continue.
```

SJF non-preemptive:

```
#include <stdio.h>
#include <stdbool.h>
struct Process {
    int pid;
    int at;
    int bt;
    int ct;
    int tat;
    int wt;
    bool finished;
};
int main() {
    int n;
    printf("Enter number of processes: ");
    scanf("%d", &n);
    struct Process p[n];
    for (int i = 0; i < n; i++) {
        p[i].pid = i + 1;
        printf("Enter arrival time and burst time of P%d: ", i + 1);
        scanf("%d %d", &p[i].at, &p[i].bt);
        p[i].finished = false;
    }
    int current_time = 0, completed = 0;
```



```

float total_tat = 0, total_wt = 0;
while (completed < n) {
    int idx = -1;
    int min_bt = 1e9;
    for (int i = 0; i < n; i++) {
        if (!p[i].finished && p[i].at <= current_time && p[i].bt < min_bt) {
            min_bt = p[i].bt;
            idx = i;
        }
    }
    if (idx == -1) {
        current_time++;
    } else {
        p[idx].ct = current_time + p[idx].bt;
        p[idx].tat = p[idx].ct - p[idx].at;
        p[idx].wt = p[idx].tat - p[idx].bt;
        total_tat += p[idx].tat;
        total_wt += p[idx].wt;
        current_time = p[idx].ct;
        p[idx].finished = true;
        completed++;
    }
}
printf("\nSJF Non-Preemptive Scheduling:\n");
printf("PID\tAT\tBT\tCT\tTAT\tWT\n");
for (int i = 0; i < n; i++) {
    printf("P%d\t%d\t%d\t%d\t%d\t%d\n",
        p[i].pid, p[i].at, p[i].bt, p[i].ct, p[i].tat, p[i].wt);
}
printf("\nAverage Turnaround Time: %.2f ms", total_tat / n);
printf("\nAverage Waiting Time: %.2f ms\n", total_wt / n);
return 0;
}

```

OUTPUT:

```
C:\Users\Umadevi\ x + v
Enter number of processes: 4
Enter arrival time and burst time of P1: 0 6
Enter arrival time and burst time of P2: 0 8
Enter arrival time and burst time of P3: 0 7
Enter arrival time and burst time of P4: 0 3

SJF Non-Preemptive Scheduling:
PID    AT    BT    CT    TAT    WT
P1      0     6     9     9     3
P2      0     8    24    24    16
P3      0     7    16    16     9
P4      0     3     3     3     0

Average Turnaround Time: 13.00 ms
Average Waiting Time: 7.00 ms

Process returned 0 (0x0)   execution time : 30.061 s
Press any key to continue.
|
```

Lab Program 2

Write a C program to simulate the following CPU scheduling algorithm to find turnaround time and waiting time.

a)Priority (pre-emptive & Non-pre-emptive)

b)Round Robin (Experiment with different quantum sizes for RR algorithm)

a)Priority (pre-emptive & Non-pre-emptive)

Priority pre-emptive:

```
#include <stdio.h>
```

```
#include <stdbool.h>
```

```
struct Process {
```

```
    int pid;
```

```
    int at;
```

```
    int bt;
```

```
    int rt;
```

```
    int pr;
```

```
    int ct;
```

```
    int tat;
```

```
    int wt;
```

```
    bool finished;
```

```
};
```

```
int main() {
```

```
    int n;
```

```
    printf("Enter number of processes: ");
```

```
    scanf("%d", &n);
```

```
    struct Process p[n];
```

```
    for (int i = 0; i < n; i++) {
```

```
        p[i].pid = i + 1;
```

```
        printf("Enter arrival time, burst time, and priority of P%d: ", i + 1);
```

```
        scanf("%d %d %d", &p[i].at, &p[i].bt, &p[i].pr);
```

```
        p[i].rt = p[i].bt;
```

```
        p[i].finished = false;
```

```
    }
```

```

int current_time = 0, completed = 0;
float total_tat = 0, total_wt = 0;

while (completed < n) {
    int idx = -1;
    int highest_priority = 1e9;

    for (int i = 0; i < n; i++) {
        if (!p[i].finished && p[i].at <= current_time && p[i].rt > 0) {
            if (p[i].pr < highest_priority) {
                highest_priority = p[i].pr;
                idx = i;
            }
        }
    }

    if (idx == -1) {
        current_time++;
    } else {
        p[idx].rt--;
        current_time++;

        if (p[idx].rt == 0) {
            p[idx].ct = current_time;
            p[idx].tat = p[idx].ct - p[idx].at;
            p[idx].wt = p[idx].tat - p[idx].bt;
            total_tat += p[idx].tat;
            total_wt += p[idx].wt;
            p[idx].finished = true;
            completed++;
        }
    }
}

printf("\nPriority Scheduling (Preemptive):\n");
printf("PID\tAT\tBT\tPR\tCT\tTAT\tWT\n");
for (int i = 0; i < n; i++) {
    printf("P%d\t%d\t%d\t%d\t%d\t%d\t%d\n",

```

```

        p[i].pid, p[i].at, p[i].bt, p[i].pr,
        p[i].ct, p[i].tat, p[i].wt);
    }

    printf("\nAverage Turnaround Time: %.2f ms", total_tat / n);
    printf("\nAverage Waiting Time: %.2f ms\n", total_wt / n);

    return 0;
}

```

OUTPUT:

```

C:\Users\Umadevi\
Enter number of processes: 5
Enter arrival time, burst time, and priority of P1: 0 4 2
Enter arrival time, burst time, and priority of P2: 1 3 1
Enter arrival time, burst time, and priority of P3: 2 5 3
Enter arrival time, burst time, and priority of P4: 3 2 2
Enter arrival time, burst time, and priority of P5: 4 4 1

Priority Scheduling (Preemptive):
PID   AT   BT   PR   CT   TAT   WT
P1     0    4    2   11   11    7
P2     1    3    1    4    3    0
P3     2    5    3   18   16   11
P4     3    2    2   13   10    8
P5     4    4    1    8    4    0

Average Turnaround Time: 8.80 ms
Average Waiting Time: 5.20 ms

Process returned 0 (0x0)   execution time : 71.753 s
Press any key to continue.

```

Priority non-preemptive:

```

#include <stdio.h>
#include <stdbool.h>
struct Process {
    int pid;
    int at;
    int bt;
    int pr;
    int ct;
    int tat;
    int wt;
    bool finished;
};

```

```

int main() {
    int n;
    printf("Enter number of processes: ");
    scanf("%d", &n);
    struct Process p[n];
    for (int i = 0; i < n; i++) {
        p[i].pid = i + 1;
        printf("Enter arrival time, burst time, and priority of P%d: ", i + 1);
        scanf("%d %d %d", &p[i].at, &p[i].bt, &p[i].pr);
        p[i].finished = false;
    }
    int current_time = 0, completed = 0;
    float total_tat = 0, total_wt = 0;
    while (completed < n) {
        int idx = -1;
        int highest_priority = 1e9;
        for (int i = 0; i < n; i++) {
            if (!p[i].finished && p[i].at <= current_time && p[i].pr <
highest_priority) {
                highest_priority = p[i].pr;
                idx = i;
            }
        }
        if (idx == -1) {
            current_time++;
        } else {
            p[idx].ct = current_time + p[idx].bt;
            p[idx].tat = p[idx].ct - p[idx].at;
            p[idx].wt = p[idx].tat - p[idx].bt;
            total_tat += p[idx].tat;
            total_wt += p[idx].wt;
            current_time = p[idx].ct;
            p[idx].finished = true;
            completed++;
        }
    }
    printf("\nPriority Scheduling (Non-Preemptive):\n");
    printf("PID\tAT\tBT\tPR\tCT\tTAT\tWT\n");
    for (int i = 0; i < n; i++) {

```

```

        printf("P%d\t%d\t%d\t%d\t%d\t%d\t%d\n",
            p[i].pid, p[i].at, p[i].bt, p[i].pr,
            p[i].ct, p[i].tat, p[i].wt);
    }
    printf("\nAverage Turnaround Time: %.2f ms", total_tat / n);
    printf("\nAverage Waiting Time: %.2f ms\n", total_wt / n);
    return 0;
}

```

OUTPUT:

```

C:\Users\Umadevi\ x + v
Enter number of processes: 5
Enter arrival time, burst time, and priority of P1: 0 3 5
Enter arrival time, burst time, and priority of P2: 2 2 3
Enter arrival time, burst time, and priority of P3: 3 5 2
Enter arrival time, burst time, and priority of P4: 6 1 1
Enter arrival time, burst time, and priority of P5: 4 4 4

Priority Scheduling (Non-Preemptive):
PID   AT   BT   PR   CT   TAT   WT
P1     0    3    5    3    3     0
P2     2    2    3   11    9     7
P3     3    5    2    8    5     0
P4     6    1    1    9    3     2
P5     4    4    4   15   11     7

Average Turnaround Time: 6.20 ms
Average Waiting Time: 3.20 ms

Process returned 0 (0x0)   execution time : 23.746 s
Press any key to continue.
|

```

Lab Program 3

Write a C program to simulate multi-level queue scheduling algorithm considering the following scenario. All the processes in the system are divided into two categories – system processes and user processes. System processes are to be given higher priority than user processes. Use FCFS scheduling for the processes in each queue.

```
#include <stdio.h>
#define MAX 10
typedef struct {
    int pid, at, bt, rt, ct, tat, wt, q;
} P;
P p[MAX];
int n, tq = 2;
int main() {
    int time = 0, completed = 0;
    float total_tat = 0, total_wt = 0;
    printf("Enter number of processes: ");
    scanf("%d", &n);
    for(int i = 0; i < n; i++) {
        p[i].pid = i + 1;
        printf("Enter AT BT Queue(1/2) for P%d: ", i+1);
        scanf("%d %d %d", &p[i].at, &p[i].bt, &p[i].q);
        p[i].rt = p[i].bt;
    }
    while(completed < n) {
        int executed = 0;
        for(int i = 0; i < n; i++) {
            if(p[i].q == 1 && p[i].rt > 0 && p[i].at <= time) {
                executed = 1;
                if(p[i].rt > tq) {
                    time += tq;
                    p[i].rt -= tq;
                } else {
```



```

        time += p[i].rt;
        p[i].rt = 0;
        p[i].ct = time;
        completed++;
    }
}
}
if(executed) continue;
for(int i = 0; i < n; i++) {
    if(p[i].q == 2 && p[i].rt > 0 && p[i].at <= time) {
        executed = 1;
        p[i].rt--;
        time++;
        if(p[i].rt == 0) {
            p[i].ct = time;
            completed++;
        }
        break;
    }
}
if(!executed) time++;
}
for(int i = 0; i < n; i++) {
    p[i].tat = p[i].ct - p[i].at;
    p[i].wt = p[i].tat - p[i].bt;
    total_tat += p[i].tat;
    total_wt += p[i].wt;
}
printf("\nProcess\tQ\tAT\tBT\tCT\tTAT\tWT\n");
for(int i = 0; i < n; i++) {
    printf("P%d\t%d\t%d\t%d\t%d\t%d\t%d\n",
        p[i].pid, p[i].q, p[i].at, p[i].bt,
        p[i].ct, p[i].tat, p[i].wt);
}
printf("\nAverage TAT = %.2f", total_tat / n);
printf("\nAverage WT = %.2f\n", total_wt / n);

```

```
    return 0;
}
```

OUTPUT:

```
C:\Users\Umadevi\ × + ▾
Enter number of processes: 4
Enter AT BT Queue(1/2) for P1: 0 4 1
Enter AT BT Queue(1/2) for P2: 0 3 1
Enter AT BT Queue(1/2) for P3: 0 8 2
Enter AT BT Queue(1/2) for P4: 10 5 1

Process Q      AT      BT      CT      TAT      WT
P1      1      0       4       6       6       2
P2      1      0       3       7       7       4
P3      2      0       8      20      20      12
P4      1     10       5      15       5       0

Average TAT = 9.50
Average WT = 4.50

Process returned 0 (0x0)   execution time : 66.757 s
Press any key to continue.
|
```

Lab Program 4

Write a C program to simulate Real-Time CPU Scheduling algorithms:

a)Rate- Monotonic

b)Earliest-deadline First

c)Proportional scheduling

a)Rate- Monotonic

```
#include <stdio.h>
#include <math.h>
int gcd(int a, int b) {
    while (b) {
        a %= b;
        int temp = a;
        a = b;
        b = temp;
    }
    return a;
}
int findLCM(int a, int b) {
    return (a * b) / gcd(a, b);
}
int main() {
    printf("1BF24CS322\n");
    int n;
    printf("Enter the number of processes:");
    scanf("%d", &n);
    int burst[n], period[n], remaining_burst[n], next_deadline[n];
    printf("Enter the CPU burst times:\n");
    for (int i = 0; i < n; i++) scanf("%d", &burst[i]);
    printf("Enter the time periods:\n");
    for (int i = 0; i < n; i++) scanf("%d", &period[i]);
    int lcm = period[0];
    for (int i = 1; i < n; i++) {
        lcm = findLCM(lcm, period[i]);
    }
    printf("LCM=%d\n\n", lcm);
    printf("Rate Monotone Scheduling:\n");
    printf("PID\tBurst\tPeriod\n");
    for (int i = 0; i < n; i++) {
```

```

    printf("%d\t%d\t%d\n", i + 1, burst[i], period[i]);
}
printf("\n");
double utilization = 0;
for (int i = 0; i < n; i++) {
    utilization += (double)burst[i] / period[i];
}
double bound = n * (pow(2.0, 1.0 / n) - 1);
printf("%f <= %f => %s\n", utilization, bound, (utilization <= bound) ? "true" :
>false");
printf("Scheduling occurs for %d ms\n\n", lcm);
for (int i = 0; i < n; i++) {
    remaining_burst[i] = 0;
    next_deadline[i] = 0;
}
int current_proc = -1;
for (int t = 0; t < lcm; t++) {
    for (int i = 0; i < n; i++) {
        if (t % period[i] == 0) {
            remaining_burst[i] = burst[i];
            next_deadline[i] = t + period[i];
        }
    }
    int selected = -1;
    for (int i = 0; i < n; i++) {
        if (remaining_burst[i] > 0) {
            if (selected == -1 || period[i] < period[selected]) {
                selected = i;
            }
        }
    }
    if (selected != current_proc) {
        if (selected != -1) {
            printf("%dms onwards: Process %d running\n", t, selected + 1);
        } else {
            printf("%dms onwards: CPU is idle\n", t);
        }
        current_proc = selected;
    }
    if (selected != -1) {
        remaining_burst[selected]--;
    }
}

```

```

    }
    return 0;
}

```

OUTPUT:

```

1BF24CS322
Enter the number of processes:2
Enter the CPU burst times:
20 35
Enter the time periods:
50 100
LCM=100

Rate Monotone Scheduling:
PID      Burst   Period
1         20      50
2         35     100

0.750000 <= 0.828427 =>true
Scheduling occurs for 100 ms

0ms onwards: Process 1 running
20ms onwards: Process 2 running
50ms onwards: Process 1 running
70ms onwards: Process 2 running
75ms onwards: CPU is idle

```

b)Earliest-deadline First

```

#include <stdio.h>
#define MAX 10
typedef struct {
    int pid;
    int burst;
    int deadline_rel;
    int period;
    int remaining_burst;
    int current_deadline;
} Task;
int main() {
    printf("1BF24CS322\n");
    Task tasks[MAX];
    int n, total_time;
    printf("Enter number of processes: ");
    if (scanf("%d", &n) != 1) return 1;
    for (int i = 0; i < n; i++) {
        tasks[i].pid = i + 1;
        printf("Enter burst, deadline, and period for Task %d: ", i + 1);

```

```

scanf("%d %d %d", &tasks[i].burst, &tasks[i].deadline_rel, &tasks[i].period);
tasks[i].remaining_burst = 0;
tasks[i].current_deadline = 0;
}
printf("Enter total scheduling time (e.g., 20): ");
scanf("%d", &total_time);
printf("\nPID\tBurst\tDeadline\tPeriod\n");
for (int i = 0; i < n; i++) {
    printf("%d\t%d\t%d\t%d\n", tasks[i].pid, tasks[i].burst, tasks[i].deadline_rel,
tasks[i].period);
}
printf("\n--- Scheduling Log ---\n");
for (int t = 0; t < total_time; t++) {
    for (int i = 0; i < n; i++) {
        if (t % tasks[i].period == 0) {
            tasks[i].remaining_burst = tasks[i].burst;
            tasks[i].current_deadline = t + tasks[i].deadline_rel;
        }
    }
    int selected = -1;
    int min_deadline = 9999;
    for (int i = 0; i < n; i++) {
        if (tasks[i].remaining_burst > 0) {
            if (tasks[i].current_deadline < min_deadline) {
                min_deadline = tasks[i].current_deadline;
                selected = i;
            }
        }
    }
    if (selected == -1) {
        printf("%dms : CPU is idle.\n", t);
    } else {
        printf("%dms : Task %d is running.\n", t, tasks[selected].pid);
        tasks[selected].remaining_burst--;
    }
}
return 0;
}

```

OUTPUT:

```
C:\Users\BMSCECSE-L4\Desktop X + - □ X
1BF24CS322
Enter number of processes: 3
Enter burst, deadline, and period for Task 1: 3 7 20
Enter burst, deadline, and period for Task 2: 2 4 5
Enter burst, deadline, and period for Task 3: 2 8 10
Enter total scheduling time (e.g., 20): 20

PID    Burst  Deadline  Period
1       3      7         20
2       2      4          5
3       2      8         10

--- Scheduling Log ---
0ms : Task 2 is running.
1ms : Task 2 is running.
2ms : Task 1 is running.
3ms : Task 1 is running.
4ms : Task 1 is running.
5ms : Task 3 is running.
6ms : Task 3 is running.
7ms : Task 2 is running.
8ms : Task 2 is running.
9ms : CPU is idle.
10ms : Task 2 is running.
11ms : Task 2 is running.
12ms : Task 3 is running.
13ms : Task 3 is running.
14ms : CPU is idle.
15ms : Task 2 is running.
16ms : Task 2 is running.
17ms : CPU is idle.
18ms : CPU is idle.
19ms : CPU is idle.
```

c)Proportional scheduling

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
#include <time.h>
```

```
typedef struct {
    char name[10];
    int tickets;
} Process;
```

```
int main() {
    int n, total_tickets = 0;
    float total_T = 0.0;
```

```

printf("Enter the number of Processes: ");
scanf("%d", &n);

Process p[n];
srand(time(NULL));

for (int i = 0; i < n; i++) {
    printf("\nProcess %d:\n", i + 1);
    sprintf(p[i].name, "P%d", i + 1);
    printf("Tickets: ");
    scanf("%d", &p[i].tickets);
    total_tickets += p[i].tickets;
    total_T += p[i].tickets;
}

printf("\n--- Proportional Share Scheduling ---\n");
printf("Enter the Time Period for scheduling: ");
int m;
scanf("%d", &m);

for (int i = 0; i < m; i++) {
    int winning_ticket = rand() % total_tickets + 1;
    int accumulated_tickets = 0;
    int winner_index = -1;

    for (int j = 0; j < n; j++) {
        accumulated_tickets += p[j].tickets;
        if (winning_ticket <= accumulated_tickets) {
            winner_index = j;
            break;
        }
    }

    printf("Tickets picked: %d, Winner: %s\n", winning_ticket,
p[winner_index].name);
}

```



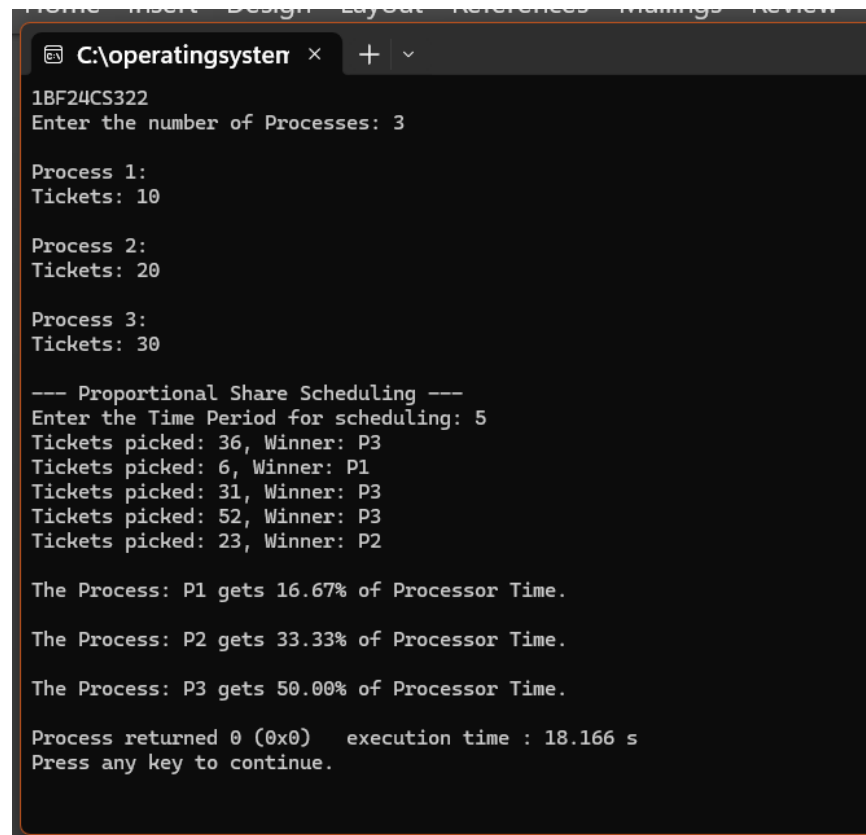
```

for (int i = 0; i < n; i++) {
    printf("\nThe Process: %s gets %.2f%% of Processor Time.\n",
        p[i].name, ((float)p[i].tickets / total_T) * 100);
}

return 0;
}

```

OUTPUT:



```

C:\operatingsystem
1BF24CS322
Enter the number of Processes: 3

Process 1:
Tickets: 10

Process 2:
Tickets: 20

Process 3:
Tickets: 30

--- Proportional Share Scheduling ---
Enter the Time Period for scheduling: 5
Tickets picked: 36, Winner: P3
Tickets picked: 6, Winner: P1
Tickets picked: 31, Winner: P3
Tickets picked: 52, Winner: P3
Tickets picked: 23, Winner: P2

The Process: P1 gets 16.67% of Processor Time.

The Process: P2 gets 33.33% of Processor Time.

The Process: P3 gets 50.00% of Processor Time.

Process returned 0 (0x0)   execution time : 18.166 s
Press any key to continue.

```

Lab Program 5

Write a C program to simulate producer-consumer problem using semaphores

```
#include <stdio.h>
#include <pthread.h>
#include <unistd.h>
#include <semaphore.h>

#define BUFFER_SIZE 5

int buffer[BUFFER_SIZE]
int in = 0, out = 0;

sem_t empty;
sem_t full;
pthread_mutex_t mutex;

void* producer(void* arg) {
    int item, i = 0;
    while (1) {
        item = i++;

        sem_wait(&empty);
        pthread_mutex_lock(&mutex);

        buffer[in] = item;
        printf("Produced: %d at buffer[%d]\n", item, in);
        in = (in + 1) % BUFFER_SIZE;

        pthread_mutex_unlock(&mutex);
        sem_post(&full);

        sleep(1);
    }
    return NULL;
```

```
}
```

```
void* consumer(void* arg) {  
    int item;  
    while (1) {  
        sem_wait(&full);  
        pthread_mutex_lock(&mutex);  
  
        item = buffer[out];  
        printf("Consumed: %d from buffer[%d]\n", item, out);  
        out = (out + 1) % BUFFER_SIZE;  
  
        pthread_mutex_unlock(&mutex);  
        sem_post(&empty);  
  
        sleep(2);  
    }  
    return NULL;  
}
```

```
int main() {  
    pthread_t prod, cons;  
  
    sem_init(&empty, 0, BUFFER_SIZE);  
    sem_init(&full, 0, 0);  
    pthread_mutex_init(&mutex, NULL);  
  
    pthread_create(&prod, NULL, producer, NULL);  
    pthread_create(&cons, NULL, consumer, NULL);  
)  
    pthread_join(prod, NULL);  
    pthread_join(cons, NULL);  
  
    sem_destroy(&empty);  
    sem_destroy(&full);  
    pthread_mutex_destroy(&mutex);
```

```
    return 0;  
}
```

OUTPUT:

```
C:\operatingsys x + v  
Produced: 0 at buffer[0]  
Consumed: 0 from buffer[0]  
Produced: 1 at buffer[1]  
Consumed: 1 from buffer[1]  
Produced: 2 at buffer[2]  
Produced: 3 at buffer[3]  
Consumed: 2 from buffer[2]  
Produced: 4 at buffer[4]  
Produced: 5 at buffer[0]  
Consumed: 3 from buffer[3]  
Produced: 6 at buffer[1]  
Produced: 7 at buffer[2]  
Consumed: 4 from buffer[4]  
Produced: 8 at buffer[3]  
Produced: 9 at buffer[4]  
Consumed: 5 from buffer[0]  
Produced: 10 at buffer[0]  
Consumed: 6 from buffer[1]  
Produced: 11 at buffer[1]  
Consumed: 7 from buffer[2]  
Produced: 12 at buffer[2]  
Consumed: 8 from buffer[3]  
Produced: 13 at buffer[3]  
Consumed: 9 from buffer[4]  
Produced: 14 at buffer[4]  
Consumed: 10 from buffer[0]  
Produced: 15 at buffer[0]  
Consumed: 11 from buffer[1]  
Produced: 16 at buffer[1]  
Consumed: 12 from buffer[2]  
Produced: 17 at buffer[2]  
Consumed: 13 from buffer[3]
```

Lab Program 6

Write a C program to simulate the concept of Dining Philosophers problem.

```
#include <stdio.h>
#include <pthread.h>
#include <unistd.h>

#define N 5 // Number of philosophers

pthread_mutex_t forks[N]; // One mutex per fork
pthread_t philosophers[N];

void* philosopher(void* num) {
    int id = *(int*)num;
    int left = id;
    int right = (id + 1) % N;

    while (1) {
        printf("Philosopher %d is thinking.\n", id);
        sleep(1);

        if (id % 2 == 0) {

            pthread_mutex_lock(&forks[left]);
            printf("Philosopher %d picked up left fork %d.\n", id, left);

            pthread_mutex_lock(&forks[right]);
            printf("Philosopher %d picked up right fork %d.\n", id, right);
        } else {

            pthread_mutex_lock(&forks[right]);
            printf("Philosopher %d picked up right fork %d.\n", id, right);

            pthread_mutex_lock(&forks[left]);
            printf("Philosopher %d picked up left fork %d.\n", id, left);
        }
    }
}
```

```

    }

    printf("Philosopher %d is eating.\n", id);
    sleep(2);

    pthread_mutex_unlock(&forks[left]);
    pthread_mutex_unlock(&forks[right]);
    printf("Philosopher %d put down forks %d and %d.\n", id, left, right);
}
return NULL;
}

int main() {
    int i;
    int ids[N];

    for (i = 0; i < N; i++) {
        pthread_mutex_init(&forks[i], NULL);
        ids[i] = i;
    }

    for (i = 0; i < N; i++) {
        pthread_create(&philosophers[i], NULL, philosopher, &ids[i]);
    }

    for (i = 0; i < N; i++) {
        pthread_join(philosophers[i], NULL);
    }

    for (i = 0; i < N; i++) {
        pthread_mutex_destroy(&forks[i]);
    }

    return 0;
}

```

OUTPUT:

```
C:\operatingsystem x + v
Philosopher 0 picked up left fork 0.
Philosopher 0 picked up right fork 1.
Philosopher 0 is eating.
Philosopher 1 put down forks 1 and 2.
Philosopher 1 is thinking.
Philosopher 4 put down forks 4 and 0.
Philosopher 4 is thinking.
Philosopher 3 picked up left fork 3.
Philosopher 3 is eating.
Philosopher 2 picked up left fork 2.
Philosopher 3 put down forks 3 and 4.
Philosopher 4 picked up left fork 4.
Philosopher 4 picked up right fork 0.
Philosopher 4 is eating.
Philosopher 3 is thinking.
Philosopher 2 picked up right fork 3.
Philosopher 2 is eating.
Philosopher 0 put down forks 0 and 1.
Philosopher 0 is thinking.
Philosopher 2 put down forks 2 and 3.
Philosopher 1 picked up right fork 2.
Philosopher 1 picked up left fork 1.
Philosopher 1 is eating.
Philosopher 0 picked up left fork 0.
Philosopher 2 is thinking.
Philosopher 4 put down forks 4 and 0.
Philosopher 4 is thinking.
Philosopher 3 picked up right fork 4.
Philosopher 3 picked up left fork 3.
Philosopher 3 is eating.
```

Lab Program 7

Write a C program to simulate Bankers algorithm for the purpose of deadlock avoidance.

```
#include <stdio.h>
#include <stdbool.h>

int main() {
    int n, m;
    printf("Enter the number of processes: ");
    scanf("%d", &n);
    printf("Enter the number of resources: ");
    scanf("%d", &m);

    int alloc[n][m], max[n][m], avail[m];
    int need[n][m];

    printf("\nEnter the Allocation Matrix:\n");
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < m; j++) {
            scanf("%d", &alloc[i][j]);
        }
    }

    printf("\nEnter the Maximum Matrix:\n");
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < m; j++) {
            scanf("%d", &max[i][j]);
        }
    }

    printf("\nEnter the Available Resources:\n");
    for (int j = 0; j < m; j++) {
        scanf("%d", &avail[j]);
    }

    // Calculate Need Matrix
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < m; j++) {
            need[i][j] = max[i][j] - alloc[i][j];
        }
    }
```



```

    }

    bool finish[n];
    for (int i = 0; i < n; i++) finish[i] = false;

    int safeSeq[n];
    int work[m];
    for (int j = 0; j < m; j++) work[j] = avail[j];

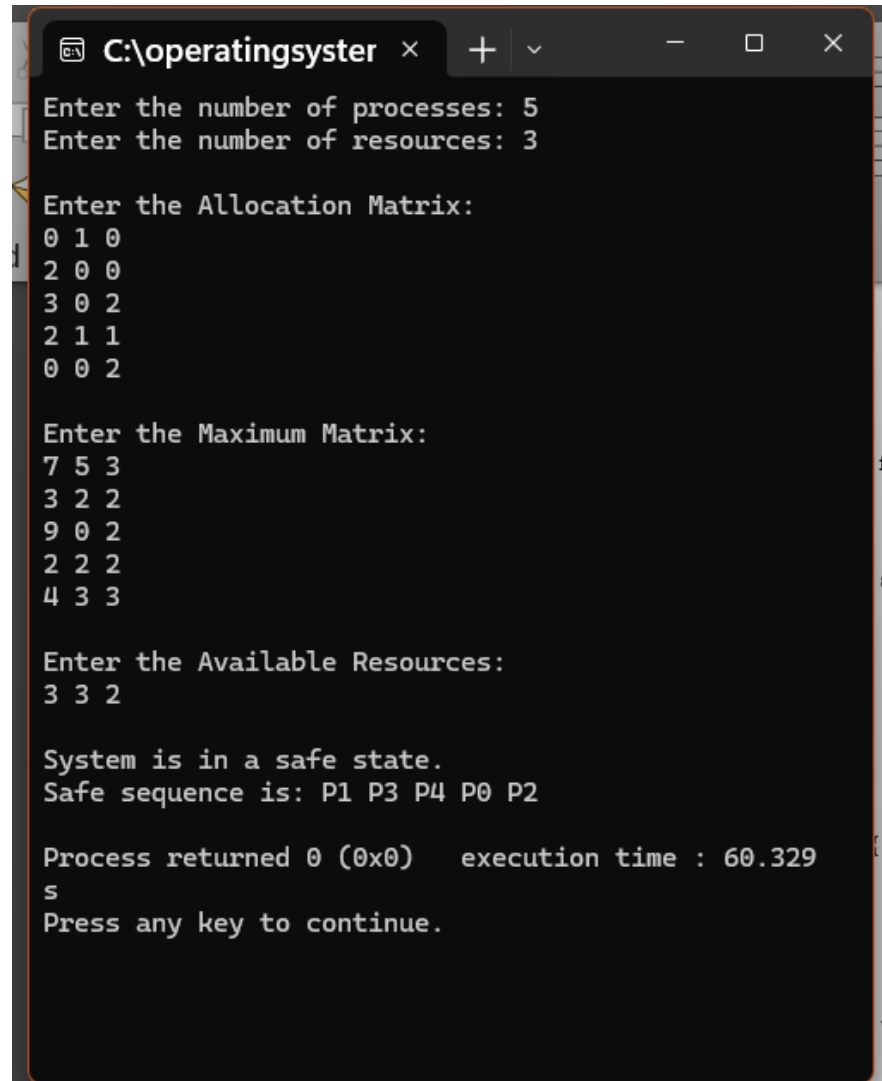
    int count = 0;
    while (count < n) {
        bool found = false;
        for (int i = 0; i < n; i++) {
            if (!finish[i]) {
                bool possible = true;
                for (int j = 0; j < m; j++) {
                    if (need[i][j] > work[j]) {
                        possible = false;
                        break;
                    }
                }
                if (possible) {
                    for (int j = 0; j < m; j++) {
                        work[j] += alloc[i][j];
                    }
                    safeSeq[count++] = i;
                    finish[i] = true;
                    found = true;
                }
            }
        }
        if (!found) {
            printf("\nSystem is not in a safe state.\n");
            return 0;
        }
    }

    printf("\nSystem is in a safe state.\nSafe sequence is: ");
    for (int i = 0; i < n; i++) {
        printf("P%d ", safeSeq[i]);
    }
    printf("\n");

```

```
    return 0;  
}
```

OUTPUT:



```
C:\operatingsystem x + - □ ×  
Enter the number of processes: 5  
Enter the number of resources: 3  
  
Enter the Allocation Matrix:  
0 1 0  
2 0 0  
3 0 2  
2 1 1  
0 0 2  
  
Enter the Maximum Matrix:  
7 5 3  
3 2 2  
9 0 2  
2 2 2  
4 3 3  
  
Enter the Available Resources:  
3 3 2  
  
System is in a safe state.  
Safe sequence is: P1 P3 P4 P0 P2  
  
Process returned 0 (0x0)   execution time : 60.329  
s  
Press any key to continue.
```